

NBA Team Performance Analysis and Game Outcome Prediction

Group Name: Team Sport Analytics

Team Members:

- Jake Crumley
- Matija Malisic
- Sumit Shrestha
- Carter Alemania

Course: CS133

Date: April 24, 2026

Abstract

This project analyzes team performance data across the 2025 and 2026 NBA seasons to determine which basketball statistics are most strongly associated with winning. Our goal was to explore whether advanced metrics such as net rating, offensive rating, assist-to-turnover ratio, and fourth-quarter scoring can help explain team success and eventually support machine learning models that predict game outcomes. To do this, we used a combination of static visualizations and interactive plots to compare teams, seasons, and game contexts.

Our visualizations showed several clear patterns. Teams with stronger offensive ratings tended to earn more wins over a season, higher assist-to-turnover ratios were often associated with wins, and fourth-quarter scoring metrics highlighted how teams performed well in high-pressure moments. These results suggest that efficiency-based statistics are meaningful indicators of success and should play an important role in any predictive model we build.

1. Background and Problem Definition

The NBA provides rich structured data for every game played throughout the season that can be used for analytics. Over the years, basketball teams and NBA analysts have grown increasingly reliant on data analytics to improve player performance, team strategy, and front-office decision making. Sports analytics as a whole has also expanded into the multi-billion-dollar sports prediction and betting industry, where complex predictive models are utilized to forecast game outcomes and player performance. While previous studies show that certain statistics correlate to winning, truly predicting game outcomes and championships remain challenging. This project

focuses on identifying measurable team statistics that explain success that can later support machine learning models for predicting both individual game outcomes, and the NBA championship outcome.

2. Dataset Overview

Three main datasets were used: `nba_team_stats_advanced.csv`, `nba_player_stats.csv`, and `nba_quarter_scores.csv`. These include team efficiency metrics, player statistics, and quarter-by-quarter scoring data. All datasets were derived directly from the NBA's API.

The `nba_team_stats_advanced.csv` dataset contains 4400 rows of advanced basketball statistics across the last couple of seasons, with each row being a different NBA game. We used the 33 different features, including team info, game results, and advanced metrics from each game, to help determine how different types of NBA team performance can affect winning games.

The `nba_player_stats.csv` dataset is a bit more expansive, containing 52766 rows of data. Each row contains player information and various statistics from a single game. The data used in this dataset was used to make generalizations about what types of players can help drive winning in the NBA.

Finally, the `nba_quarter_scores.csv` dataset contains 4000 rows of NBA game data, with each row describing how performance varied for each team throughout a single game. This data was used to try and find any scoring patterns that related to winning games in the NBA.

Additionally, we used a supplementary `TeamStatisticsAdvanced` dataset early on from Kaggle to explore some more historical NBA data. This data helped us set a baseline of what to look for once we moved on to the NBA API datasets.

3. Analytical Questions

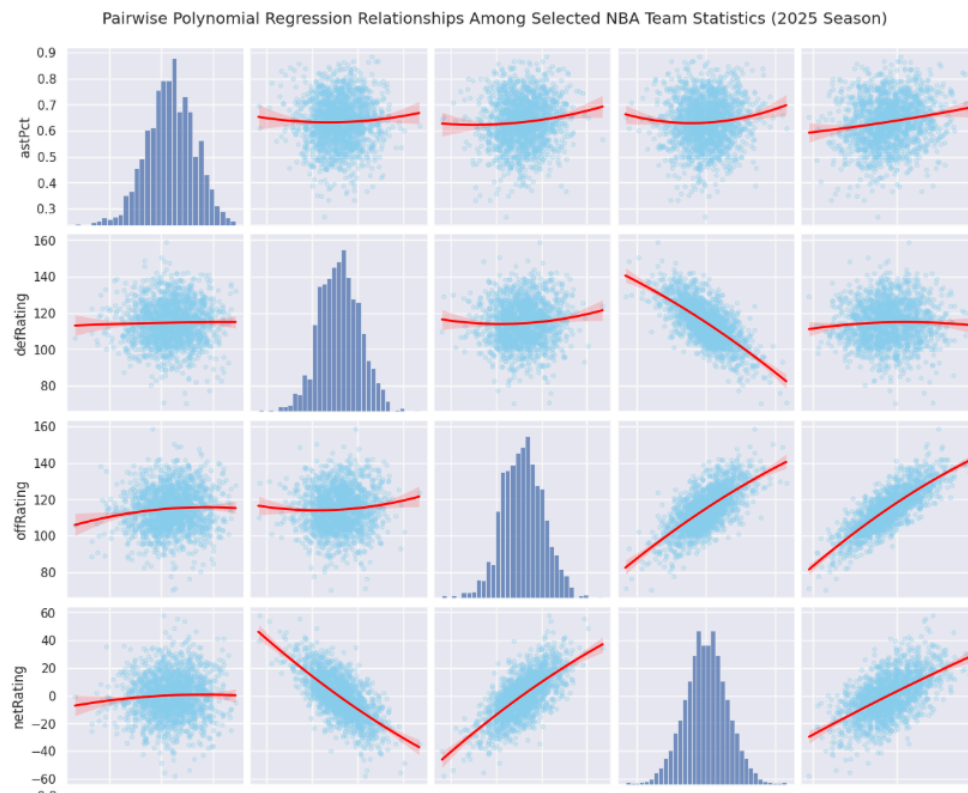
We used these five analytical questions to help guide our research throughout this project:

1. Do different types of team rating (net rating, offensive rating, etc.) correlate to winning?
2. How does 4th-quarter performance affect game outcomes?
3. What stats, such as assist-to-turnover ratio, help boost offensive rating the most?
4. Do teams with higher pace tend to have higher net ratings?
5. Does the data from the 2026 NBA strongly point to a specific team winning the NBA Finals?

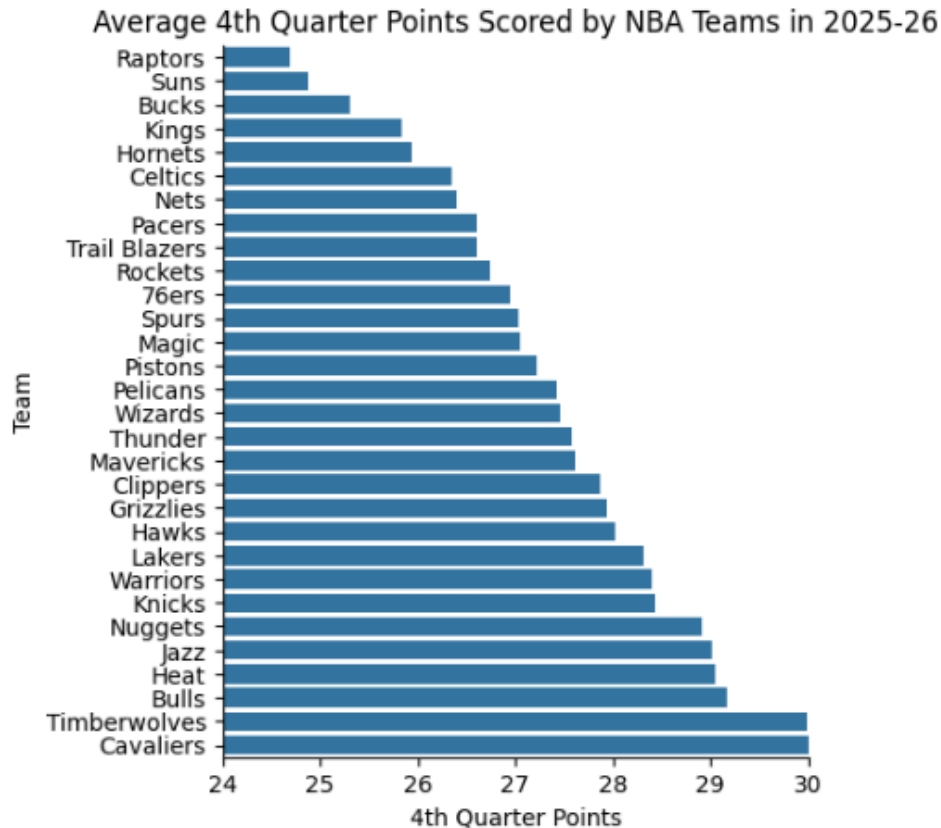
4. Data Cleaning and Preparation

In our initial data exploration, we noticed that our data was pretty well formatted due to it coming straight from the NBA's API. We did have some missing data values to deal with, but these values mostly came from columns that were not necessary for our data analysis, such as player position or college (positions are very fluid in basketball and many players play multiple positions, and college is irrelevant to their professional performance). Overall, any columns with invalid or missing data didn't really need to be used anyways, and our data was already shaped in a way that was preferable for the type of analysis we wanted to proceed with.

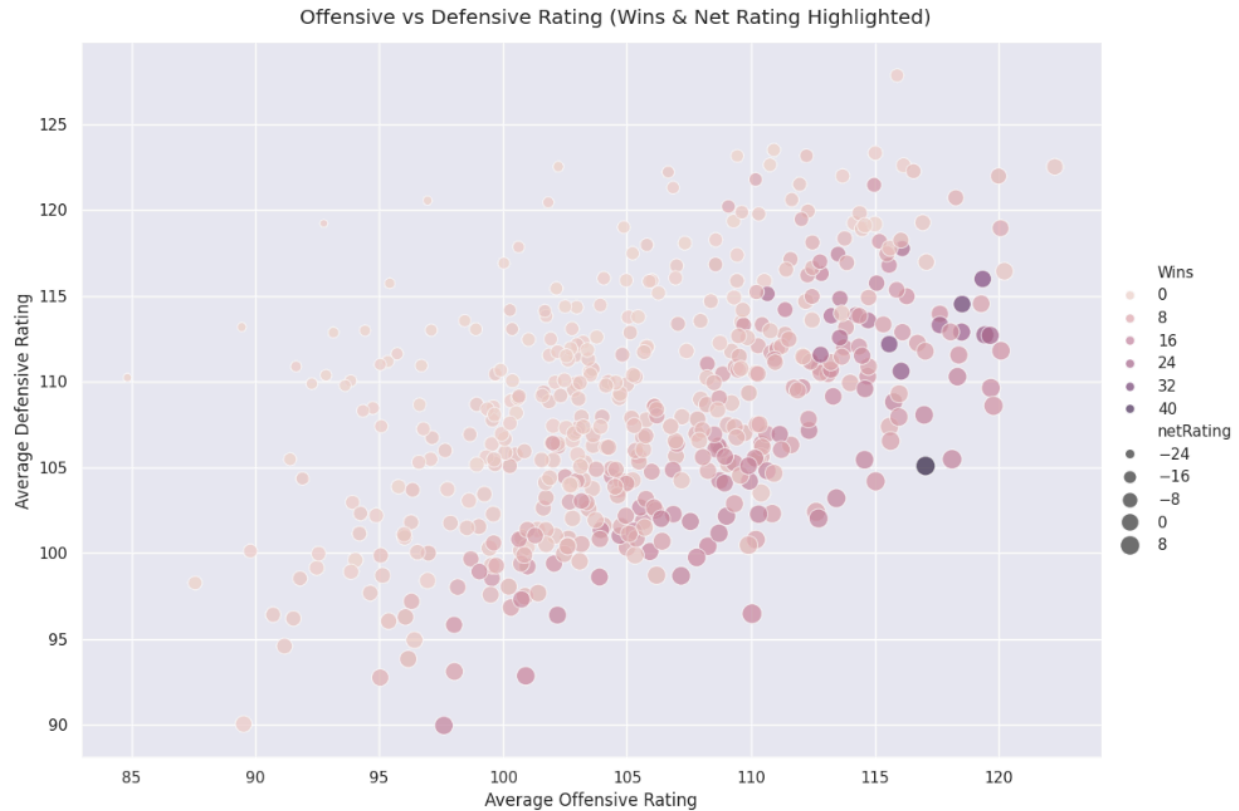
5. Static Visualization Findings



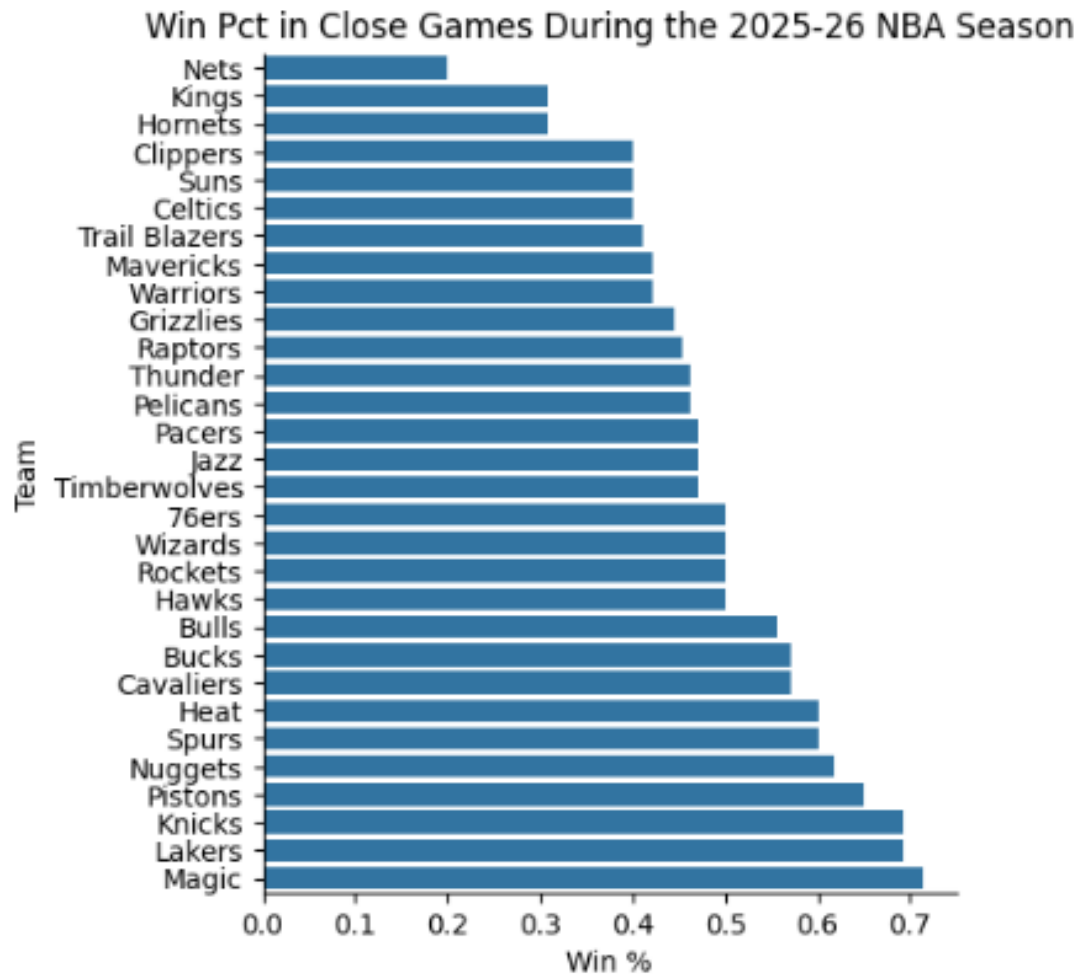
This pairwise polynomial regression chart (whole chart not depicted here) helped us determine if there were any specific stats that had direct correlations that we could further explore down the line.



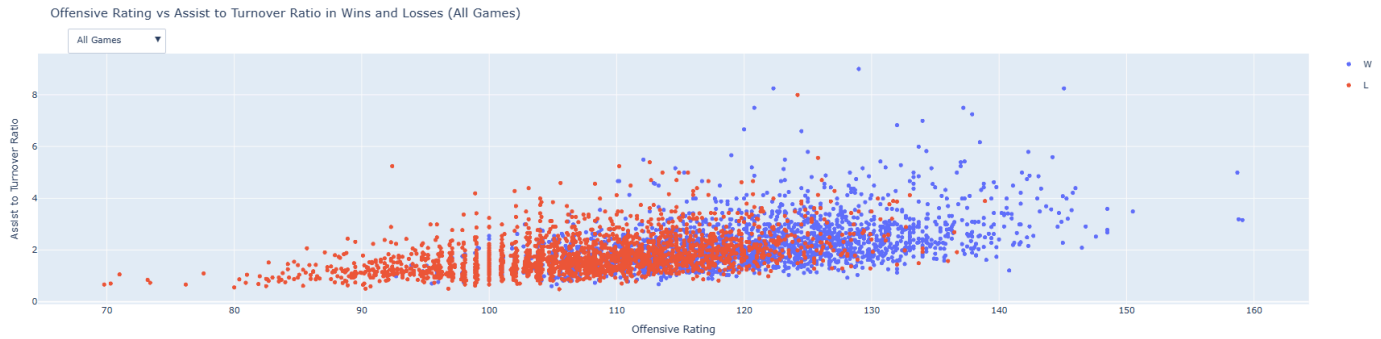
This bar chart from P2 depicts the average number of 4th quarter points scored by different NBA teams in the current season (the season we want to predict the finals winner for). One thing that we noticed is that the two teams with the highest values (by a large margin), the Cavaliers and Timberwolves, both ended up being high seeds in the NBA playoffs, which just started about a week ago. On the other hand, looking at the teams with the lowest values, four of the lowest 5 all missed the playoffs entirely, while the other team that made the playoffs snuck in as the last seed in their conference. One thing that should be noted is that despite this correlation, there is an exception with certain teams and where they lie on this chart. Out of the top four seeded teams in the NBA playoffs, not one of them is in the top ten in 4th quarter points scored on average. However, this may be explained by those teams' dominance before the fourth quarter. The best teams in the league may not have to go all out in the fourth quarter if they have already accumulated a large lead in that game, which may lead to their fourth-quarter scoring taking a dip.



This chart from P1 shows the relationship between average offensive and defensive rating for different NBA teams in specific games, and how those metrics relate to winning and total net rating. One thing that should be explained is how these metrics are inversely related: while having a higher offensive rating is typically seen as good, the opposite is true with defensive rating, as teams typically want a lower defensive rating. Net rating is a combination of these two ratings, as a team with a good net rating often has a high offensive rating and a low defensive rating. This can be seen in the chart, as the larger points tend to be in that range. We can also see that winning strongly correlates with a high offensive rating, a low defensive rating, and an overall better net rating in this chart. After visualizing this, we will be able to heavily factor net rating into our prediction for the NBA playoffs.

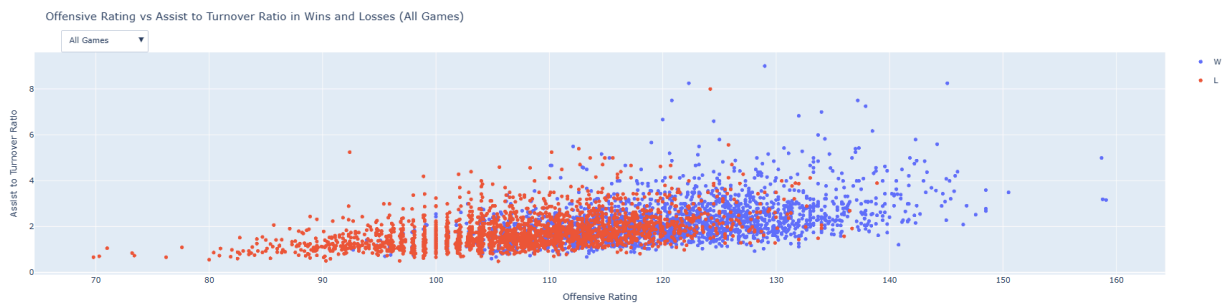


This is another categorical chart from P2 that depicts each NBA team's winning percentage in 'close games', or games decided by less than five points, in the current season. In such a highly competitive league, this metric can help us determine which teams will succeed in making the playoffs, as well as which teams will do well during the playoffs. This chart has an even stronger correlation than the first categorical chart, as seven of the best eight teams (including the top six) are teams that made the playoffs, while the bottom four teams all missed the playoffs. With the NBA playoffs being so competitive and game results being marginal, this metric will certainly factor into our playoff prediction.

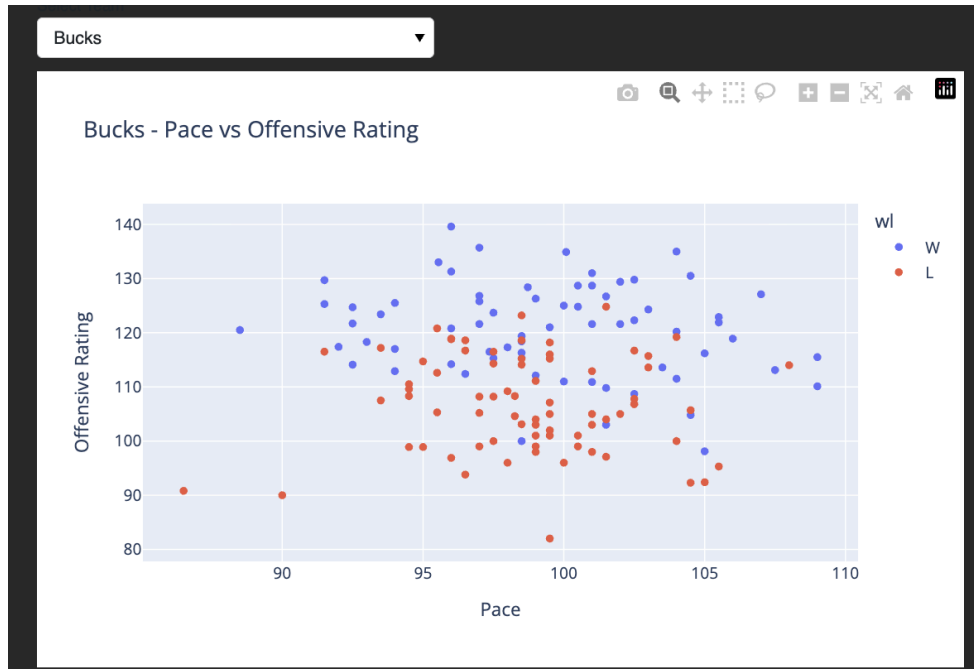


This chart (which also happened to be one of our interactive charts, but was essential to our analysis) depicts how offensive rating relates to a team's assist-to-turnover ratio, and how these stats relate to winning. We can clearly see here that winning clusters are associated with a higher offensive rating, and that there is a positive correlation between offensive rating and assist-to-turnover ratio. Because we were able to see this relationship, we were able to help determine that being efficient on offense helps improve overall offensive output, so the assist-to-turnover ratio is another thing we will take into account when predicting our finals winner.

6. Interactive Visualization Component



This chart is an interactive chart that depicts how offensive rating relates to a team's assist-to-turnover ratio, and how these stats relate to winning. We can clearly see here that winning clusters are associated with a higher offensive rating, and that there is a positive correlation between offensive rating and assist-to-turnover ratio. Because we were able to see this relationship, we were able to help determine that being efficient on offense helps improve overall offensive output, so the assist-to-turnover ratio is another thing we will take into account when predicting our finals winner.



This chart is another interactive plot we've created. It visualizes the relationship between team pace and offensive rating for the selected team. Each point represents a game, with color indicating the outcome (win or loss), allowing users to quickly distinguish performance differences by result. The visualization highlights how variations in pace may influence efficiency, making trends and outliers easier to identify. The dashboard includes interactive controls, such as a team selection dropdown. These interactions are purposeful, allowing users to compare performance across scenarios and better understand the relationship between key metrics. Clear setup and execution instructions are provided in the README. There is no need for pip installation or virtual environments, as we used Google Colab for everything, so only the cells need to be run in order.

7. Machine Learning Pipeline and Evaluation

I. Machine Learning Models

Three classification models were implemented for this project: Logistic Regression, Random Forest, and Support Vector Classifier (SVC).

Before training the models, the dataset was preprocessed to prepare it for classification. We selected the most relevant numerical features from the NBA team statistics dataset, including `offRating`, `defRating`, `pace`, `astTo`, `rebPct`, `tsPct`, and `efgPct`. The target variable, `wl`, was encoded

into binary form, where a win was represented as 1, and a loss was represented as 0. Any rows with missing values in the selected features were removed so that the models would be trained on clean and complete data.

The data was then split into training and testing sets using an 80/20 ratio. A stratified split was used to preserve the balance of wins and losses in both sets. For each model, a pipeline was created that included StandardScaler for normalization, followed by the machine learning model used for prediction. This ensured that the same preprocessing steps were applied consistently across all three models.

II. Model Training and Selection

To evaluate the models fairly, 5-fold stratified cross-validation was applied to the training data. This method allowed us to assess model performance across multiple data splits rather than relying on a single validation result.

The models were compared using the following evaluation metrics:

- Accuracy
- F1-score
- ROC-AUC

After cross-validation, the average performance of each model was compared. The model selection strategy was based primarily on ROC-AUC, and the model with the highest ROC-AUC score was chosen as the best-performing model. ROC-AUC was selected because it provides a strong overall measure of classification performance by balancing the true positive rate and false positive rate.

III. Model Evaluation

After selecting the best-performing model, it was trained on the full training dataset and then evaluated on the held-out test set. The final model was assessed using the following metrics:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

A confusion matrix was also used to visualize the prediction results and show how many wins and losses were correctly or incorrectly classified. Overall, the best model achieved high accuracy with very few misclassifications. This suggests that team-level statistics such as offensive rating, defensive rating, pace, and shooting efficiency are strong predictors of NBA game outcomes. These results align with earlier exploratory data analysis, where strong relationships between performance metrics and game outcomes were observed.

8. Results, Code Output, and Interpretation

The machine learning pipeline was implemented in Python using the scikit-learn library. Key steps of the implementation are shown below.

I. Data Preparation

```
# Defining features and target variable
feature_cols = ['offRating', 'defRating', 'pace', 'astTo', 'rebPct', 'tsPct', 'efgPct']
target_col = 'wl'

# Dropping records with missing feature or target data
ml_data = team_stats[feature_cols + [target_col]].dropna()

# Encoding target to binary (Win: 1, Loss: 0)
X = ml_data[feature_cols]
y = ml_data[target_col].map({'W': 1, 'L': 0})
```

- Selected relevant features
- Converted win/loss into binary values
- Removed missing data

II. Train-Test Split

```
# Applying stratified 80/20 split to preserve class distribution
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
print(f"Data split: {X_train.shape[0]} training samples, {X_test.shape[0]} testing samples.\n")
```

- Used 80/20 split
- Applied stratified sampling to maintain class balance

III. Model Pipelines

```
# Constructing pipelines with standard scaling (required for SVC and Logistic Regression convergence)
pipeline_lr = Pipeline([('scaler', StandardScaler()), ('lr', LogisticRegression(random_state=42))])
pipeline_rf = Pipeline([('scaler', StandardScaler()), ('rf', RandomForestClassifier(random_state=42))])
pipeline_svc = Pipeline([('scaler', StandardScaler()), ('svc', SVC(probability=True, random_state=42))])
```

- Combined preprocessing and model training
- Ensures consistent scaling across models

IV. Cross-validation

```
# Executing 5-fold stratified CV for robust metric estimation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_results = []

for name, model in models.items():
    scores = cross_validate(model, X_train, y_train, cv=cv,
                           scoring=['accuracy', 'f1', 'roc_auc'])

    cv_results.append({
        'Model': name,
        'CV Mean Accuracy': scores['test_accuracy'].mean(),
        'CV Mean F1': scores['test_f1'].mean(),
        'CV Mean ROC-AUC': scores['test_roc_auc'].mean()
    })
```

- Used 5-fold stratified cross-validation
- Evaluated using multiple metrics

V. Model selection and evaluation

```
# Selecting best model architecture
best_model_name = cv_df.sort_values(by='CV Mean ROC-AUC', ascending=False).iloc[0]['Model']
best_model = models[best_model_name]
print(f"Best model selected: {best_model_name}\n")

print("--- 3. Model Evaluation ---")

# Fitting selected model on the entire training set
best_model.fit(X_train, y_train)

# Generating predictions on hold-out set
y_pred = best_model.predict(X_test)
y_proba = best_model.predict_proba(X_test)[:, 1]

# Computing evaluation metrics
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_proba)
```

```

target_names = ['Loss', 'Win']
print("Classification Report:")
print(classification_report(y_test, y_pred, target_names=target_names))

# Rendering confusion matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=target_names)
disp.plot(cmap='Blues', values_format='d')
plt.title(f'Confusion Matrix - {best_model_name}')
plt.show()

```

- Selected the best model based on ROC-AUC score
- Evaluated performance on the test set
- Generated a confusion matrix

Overall, the code snippet shows the evaluation process for the selected machine learning model. After training, predictions are generated on the test dataset and used to compute performance metrics such as accuracy, precision, recall, F1-score, and ROC-AUC as seen below. Additionally, a confusion matrix is created and visualized to provide a clear breakdown of the correct and incorrect predictions.

Full code at: [term-project-sports-analytic](#)

Output:

```

--- 1. Data Ingestion & Preprocessing ---
Data split: 3520 training samples, 880 testing samples.

--- 2. Model Training & Cross-Validation ---
Cross-Validation Results:
      Model  CV Mean Accuracy  CV Mean F1  CV Mean ROC-AUC
Logistic Regression      0.988920    0.988924    0.999616
Random Forest           0.980114    0.980039    0.998467
SVC                     0.981534    0.981565    0.998978

Model Selection Strategy: Maximizing CV ROC-AUC.

Best model selected: Logistic Regression

--- 3. Model Evaluation ---
Test Set Performance (Logistic Regression):
Accuracy: 0.9898
Precision: 0.9954
Recall: 0.9841
F1 Score: 0.9897
ROC AUC: 0.9994

Classification Report:
      precision    recall  f1-score   support

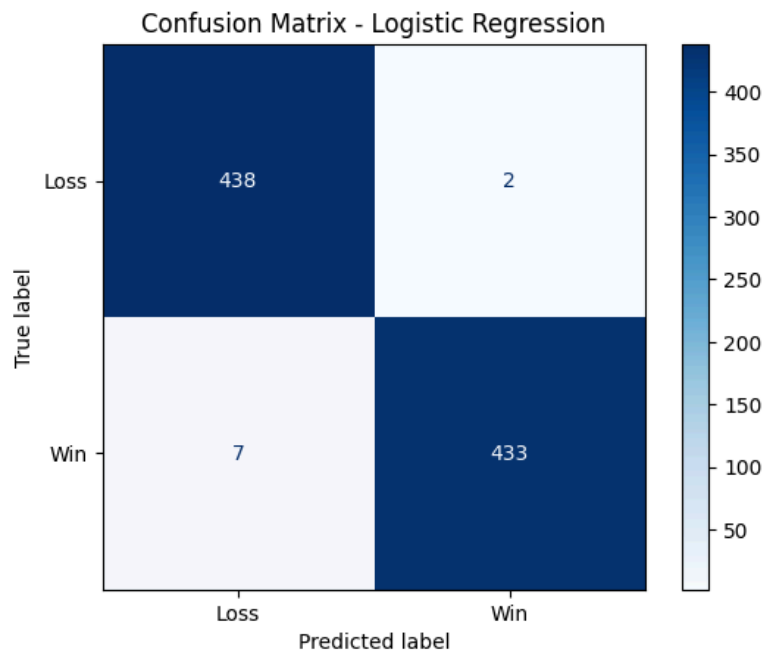
   Loss       0.98       1.00       0.99        440
    Win       1.00       0.98       0.99        440

 accuracy          0.99
 macro avg          0.99
weighted avg          0.99

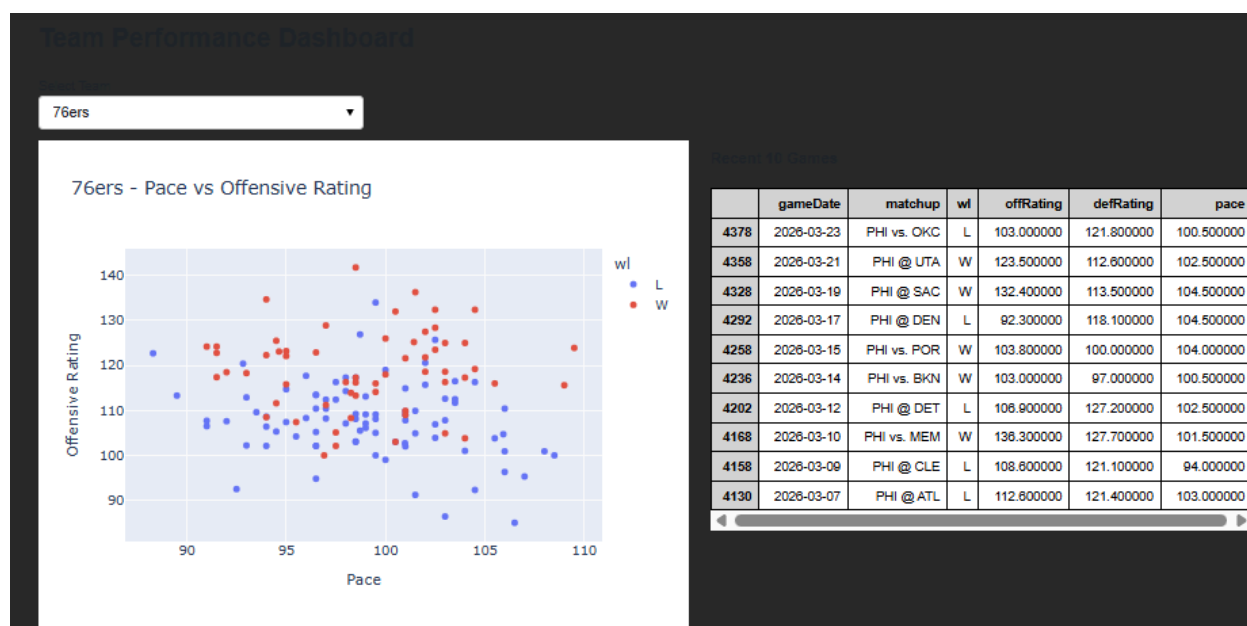
```

This classification report provides a detailed summary of the model's performance across both classes (wins and losses). The results show very high precision, recall, and F1-scores (approximately 0.99). Indicating that the model is highly accurate and consistent in its predictions. Precision reflects how many predicted wins/losses

were correct, while recall measures how well the model captured all actual wins/losses. The high value across all metrics suggests that the model is not biased toward one class and performs well overall. The results also demonstrated that the selected features, such as offensive rating, defensive rating, and pace, are strong predictors of game outcomes, directly supporting the project's objective.



This confusion matrix visualizes the model's prediction performance by comparing outcomes with predicted outcomes. The majority of values lie along the diagonal, representing correct classifications, while only a small number of instances fall outside the diagonal, indicating misclassifications. Specifically, the model made only 9 incorrect predictions out of 880 total games, which is extremely good, and it demonstrates a very low error rate. Both wins and losses are predicted with high accuracy, showing that the model generalizes well and performs reliably across different outcomes.



This interactive plot visualizes the relationship between team pace and offensive rating, with each point representing a game and color indicating whether the game was a win or loss. A clear pattern emerges where games with higher offensive ratings are more frequently associated with wins, suggesting that offensive efficiency is a key factor in determining success. In contrast, pace does not show a strong linear relationship with outcomes, indicating that playing faster alone does not guarantee better performance. Instead, efficiency metrics appear to have a greater impact. This insight connects directly to the original hypothesis we had, showing that while multiple features were considered, offensive effectiveness plays a more important role in predicting game outcomes than pace alone.

9. Reproducibility, GitHub, and Documentation

The GitHub repository includes all project components:

- Jupyter notebooks for data exploration and visualization
- The machine learning implementation
- Dataset files stored in the db/ folder

The project is organized into clear folders:

- Project Assignments/ for notebooks
- db/ for datasets

The code is structured and includes comments to explain key steps in the pipeline.

The project can be reproduced by:

- Installing required Python libraries (e.g., pandas, scikit-learn, matplotlib)
- Running the notebooks or the machine learning script

A README file is included to describe the project structure and provide basic setup instructions.

12. Conclusion

Overall, this project showed that NBA game outcomes can be predicted reasonably well using team-level efficiency metrics. Although we scoped down the MVP and Finals prediction questions, the final pipeline still answered the core goal of building a game-level win/loss predictor. The results show that statistics like offensive rating, defensive rating, pace, and other efficiency-based features can capture meaningful differences between teams.

While the project does not fully solve season-level forecasting, it creates a strong foundation for future work. A natural next step would be adding player-level data for MVP prediction and using the game-level model inside a playoff Monte Carlo simulation for Finals forecasting. In the end, our pipeline demonstrates that NBA outcomes are learnable at the game level and can be expanded into more advanced season level predictions.

References

Kaggle dataset: Historical NBA Data and Player Box Scores

Libraries used: pandas, seaborn, matplotlib, plotly, panel, numpy, geopy